

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №3

по дисциплине «Проектирование защищенных интеллектуальных
информационных систем»

на тему «Анализ безопасности кода.

Часть 1. Анализ безопасности кода разработанного приложения»

Выполнил студент гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Сальников Д. А.

Минск 2026

Цель: получить практические навыки анализа безопасности кода приложения и состояния его оперативной памяти – в дополнение к анализу «чёрного ящика» из предыдущих лабораторных работ – и на практике проверить, как ведут себя конфиденциальные и неконфиденциальные данные в оперативной памяти процесса на всех стадиях их жизненного цикла: при создании, обновлении и удалении.

Задание:

а) Разработать приложение с интерактивным пользовательским интерфейсом, позволяющим создать, обновить и удалить данные в/из оперативной памяти – как конфиденциальные, так и неконфиденциальные; конфиденциальные данные после ввода должны быть зашифрованы или захешированы.

б) Выполнить анализ безопасности разработанного приложения с точки зрения кода.

в) Выполнить анализ оперативной памяти и процессорного времени приложения.

г) Создать дампы оперативной памяти приложения после ввода, после обновления и после удаления данных; выполнить анализ дампов, уделив особое внимание работе с конфиденциальными данными.

д) Оформить отчёт о проделанной работе.

Постановка задачи. Вместо разработки нового приложения с нуля существующее приложение **SecureApp (Lab2.1-SecureApp)** расширено новой возможностью: разделом «In-memory data», данные которого умышленно никогда не сохраняются на диск (не через Entity Framework Core, не через файл, не через какой-либо кэш) и живут исключительно в памяти процесса backend-сервера до его перезапуска. Раздел, как и требует задание, поддерживает создание, обновление и удаление как конфиденциальных данных (шифруемых непосредственно перед помещением в оперативную память), так и неконфиденциальных (храняемых в открытом виде) – через тот же веб-интерфейс на React, что и постоянные данные лабораторной работы №2.1, но на отдельных вкладках. Такое расширение позволяет исследовать в чистом виде именно оперативную память процесса, не смешивая её с содержимым файла базы данных, которое уже было предметом анализа в лабораторной работе №2.1.

Краткое описание разработанного приложения

В backend (**SecureApp.Api**) добавлен класс **InMemoryRecordStore** – синглтон-сервис, хранящий записи в двух **ConcurrentDictionary**

(для конфиденциальных и неконфиденциальных данных) без единого обращения к Entity Framework Core, файловой системе или иному постоянному хранилищу; идентификаторы записей выдаются атомарно через `Interlocked.Increment`, что делает сервис безопасным для параллельных запросов без явных блокировок. Поверх сервиса добавлены два REST-контроллера, `InMemoryConfidentialController` и `InMemoryPublicController`, идентичные по контракту (маршруты, авторизация, разбиение по владельцу) контроллерам постоянных данных из лабораторной работы №2.1, но обслуживающие маршруты `/api/memory/confidential` и `/api/memory/public`. На фронтенде добавлены две вкладки «In-memory confidential» и «In-memory public», использующие тот же компонент таблицы с поиском, что и постоянные данные, — различие видно пользователю только по надписи «RAM only – not persisted, lost on restart» на соответствующей вкладке.

Конфиденциальные данные шифруются алгоритмом AES-256-GCM (тем же классом `ConfidentialCrypto`, что и в лабораторной работе №2.1) непосредственно в момент создания или обновления записи — то есть в структурах данных самого сервиса никогда не хранится ни одной конфиденциальной строки в открытом виде, только зашифрованные байты (листинг 1).

```
1 public ConfidentialEntry CreateConfidential(int ownerId, string title, string content)
2 {
3     var id = Interlocked.Increment(ref _nextConfidentialId);
4     var now = DateTime.UtcNow;
5     var entry = new ConfidentialEntry(id, ownerId, Encrypt(title), Encrypt(content),
6     ↪ now, now);
7     _confidential[id] = entry;
8     return entry;
9 }
```

Листинг 1 – Создание записи: шифрование выполняется до попадания данных в словарь

Анализ защищённости приложения с точки зрения кода

1. Обработка невалидных данных

Семь видов заведомо некорректного ввода были поданы на новый маршрут напрямую (см. листинг 2): значения не того типа (числа и булево значение вместо строк), синтаксически некорректный JSON, отсутствие обязательного поля, превышение ограничения длины поля (10 000 символов), управляющие символы (перевод строки, символ табуляции) внутри значения и полностью пустое тело запроса. Все семь случаев отклонены кодом 400 **Bad Request** системой валидации ASP.NET Core, кроме управляющих символов внутри значения – они являются полноправным содержимым текстового поля, а не структурной ошибкой, и корректно приняты (201 **Created**). После всех проверок контрольный запрос к тому же маршруту подтвердил, что процесс не аварийно завершился и не завис – ни одно из некорректных значений не привело к необработанному исключению на уровне процесса, а осталось изолированным в рамках обработки одного HTTP-запроса.

```
1 == Fault-injection test: /api/memory/confidential ==
2 [PASS] title/content as wrong JSON types (number/bool instead of string) ->
   ↳ HTTP 400
3 [PASS] syntactically malformed JSON body -> HTTP 400
4 [PASS] missing required field ('content' absent) -> HTTP 400
5 [PASS] oversized content (10050 chars, limit is 10000) -> HTTP 400
6 [PASS] control characters inside content (\n, \t) accepted as data -> HTTP
   ↳ 201
7 [PASS] empty request body -> HTTP 400
8 [PASS] process still alive and responsive after all 6 malformed requests ->
   ↳ HTTP 200
9 === Summary ===
10 Checks: 7 total, 0 crashes, 0 hangs
```

Листинг 2 – Реакция на некорректный ввод: 7 проверок, 0 падений процесса

2. Работа с конфиденциальными данными в коде

В методы `ConfidentialCrypto.Encrypt/Decrypt` добавлена явная очистка собственных промежуточных буферов (`Array.Clear(plainBytes)`) сразу после использования, в блоке `finally` – то есть даже при исключении внутри самого шифрования байтовый буфер не останется висеть в памяти дольше необходимого. Это осознанно называется в коде и в этом отчёте *мерой «наилучшего возможного» (best-effort)*, а не гарантией: строка `string` в

.NET неизменяема, и исходная строка с открытым текстом, переданная методу вызывающим кодом (в том числе десериализатором JSON, создавшим её из тела HTTP-запроса), никаким программным способом самим методом очищена быть не может – он контролирует только свои собственные, дополнительно созданные буферы. Управляемая среда исполнения (.NET CLR) в этом отношении принципиально отличается от кода на C/C++, где буфер с паролем можно явно обнулить сразу после использования и быть уверенным, что других копий не осталось: в управляемой среде таких гарантий нет, и, как показано далее в разделе «Анализ дампов оперативной памяти», это не абстрактная угроза, а наблюдаемое на практике поведение. По этой же причине класс **System.Security.SecureString** сознательно не используется: начиная с .NET Core он не даёт защиты вне Windows и, что важнее, не защищает от ровно той угрозы, которая здесь актуальна – копий строки, уже созданных инфраструктурой ASP.NET Core до попадания данных в код приложения.

3. Потокбезопасность

Поскольку все операции доступны нескольким одновременным HTTP-запросам, хранилище построено на **ConcurrentDictionary** вместо обычного **Dictionary** с ручной блокировкой, а выдача идентификаторов – на атомарном **Interlocked.Increment** вместо чтения и увеличения обычного поля, что исключает состояние гонки (два запроса, получившие один и тот же идентификатор) без единого явного **lock** в коде сервиса.

Анализ оперативной памяти и процессорного времени приложения

Оперативная память и загрузка процессора процесса backend измерялись инструментом `dotnet-counters` (аналог `jvisualvm` для .NET) в двух режимах: в состоянии покоя и под нагрузкой в 900 последовательных HTTP-запросов (300 циклов создание → обновление → удаление записи) за приблизительно 10 секунд.

- **Резидентная память (`working set`).** В состоянии покоя – около 259 МБ; на пике нагрузки – около 293 МБ (прирост ≈ 34 МБ на 900 операций), что подтверждает: временные объекты создаются и достаточно быстро собираются сборщиком мусора, без неограниченного роста памяти.

- **Процессорное время.** В состоянии покоя процесс потреблял около 0,13 с пользовательского времени CPU на секунду наблюдения ($\approx 13\%$ одного ядра); под нагрузкой – до 1,09 с на секунду (то есть свыше одного полностью загруженного ядра, за счёт нескольких потоков одновременно), возвращаясь к уровню покоя сразу после завершения нагрузки.

- **Скорость выделения памяти в куче (`allocation rate`).** От 0,7 до 1,7 МБ в секунду под нагрузкой – ожидаемо для рабочей нагрузки, где на каждый HTTP-запрос выделяются короткоживущие объекты (строки из JSON, объекты записей).

- **Пропускная способность пула потоков.** До 877 завершённых рабочих элементов в секунду – согласуется с 900 запросами, реально обработанными за ≈ 10 секунд.

- **Паузы сборщика мусора.** На гранулярности в 1 секунду не зафиксировано значимых пауз сборщика мусора – нагрузка данного масштаба не вызывает заметных для пользователя задержек ответа из-за GC.

Совокупно приложение демонстрирует управляемый рост потребления ресурсов под нагрузкой без признаков утечки памяти или чрезмерной загрузки процессора для нагрузки такого масштаба.

Анализ дампов оперативной памяти

Дампы снимались утилитой `dotnet-dump collect` (аналог `jstack`/полного дампа кучи JVM) в трёх контрольных точках, предписанных заданием: после создания, после обновления и после удаления одной конфиденциальной и одной неконфиденциальной записи, каждая с уникальным маркером в содержимом (например, `CONFIDENTIAL-PLAINTEXT-MARKER-AXQ7391`). Перед третьим дампом дополнительно принудительно вызывалась полная сборка мусора – побочный эффект команды `dotnet-gcdump collect`, которая перед снятием дампа кучи запрашивает у среды выполнения блокирующую сборку, – чтобы проверить, действительно ли удаление записи из структуры данных приложения приводит к исчезновению значения из физической памяти процесса после того, как сборщик мусора получил шанс отработать. Каждый дамп затем анализировался утилитой `strings` с поиском обоих маркеров – методика, аналогичная поиску секретов в файле базы данных из лабораторной работы №2.1, но применённая к дампу процесса, а не к файлу на диске.

```
1 == Dump 1: after CREATE (confidential id=2, public id=2) ==
2 CONFIDENTIAL-PLAINTEXT-MARKER-AXQ7391 (confidential content, V1): 3 occurrence(s)
3 PUBLIC-PLAINTEXT-MARKER-BXQ7391 (public content, V1):          3 occurrence(s)
4
5 == Dump 2: after UPDATE (V1 -> V2 on both records) ==
6 CONFIDENTIAL-PLAINTEXT-MARKER-AXQ7391 (old V1, logically replaced): 3 occurrence(s)
7 CONFIDENTIAL-PLAINTEXT-MARKER-V2-CYT8402 (new V2, current):        3 occurrence(s)
8 PUBLIC-PLAINTEXT-MARKER-BXQ7391 (old V1, logically replaced):       3 occurrence(s)
9 PUBLIC-PLAINTEXT-MARKER-V2-DYT8402 (new V2, current):              3 occurrence(s)
10
11 == Dump 3: after DELETE of both records + forced full GC (dotnet-gcdump collect) ==
12 CONFIDENTIAL-PLAINTEXT-MARKER-AXQ7391 (V1, deleted 2 versions ago): 3 occurrence(s)
13 CONFIDENTIAL-PLAINTEXT-MARKER-V2-CYT8402 (V2, deleted):           3 occurrence(s)
14 PUBLIC-PLAINTEXT-MARKER-BXQ7391 (V1, deleted 2 versions ago):      3 occurrence(s)
15 PUBLIC-PLAINTEXT-MARKER-V2-DYT8402 (V2, deleted):                 3 occurrence(s)
```

Листинг 3 – Число вхождений маркеров в каждом из трёх дампов

Результат оказался одинаковым во всех трёх контрольных точках и для обеих версий значения (до и после обновления): и конфиденциальный, и неконфиденциальный маркер найдены в дампе процесса по три раза каждый – включая дамп, снятый после удаления записи и принудительной полной сборки мусора. Иначе говоря, ни шифрование данных перед сохранением в структуру приложения, ни последующее удаление записи, ни явно запрошенная сборка мусора не убрали открытый текст конфиденциального значения из памяти процесса.

Это не свидетельствует об ошибке именно в коде хранилища (`InMemoryRecordStore` нигде не хранит открытый текст – в его собственных

структурах данных лежат только зашифрованные байты, что было подтверждено отдельно на уровне кода в предыдущем разделе). Наиболее вероятное объяснение, согласующееся с внутренним устройством ASP.NET Core: конвейер обработки HTTP-запроса (Kestrel и сериализатор `System.Text.Json`) для производительности активно использует переиспользуемые буферы (`ArrayPool<byte>/ArrayPool<char>` из общего пула), в которые копируется тело запроса при разборе JSON и тело ответа при его формировании. Буфер, возвращённый в пул, не обнуляется и с точки зрения сборщика мусора не является мусором вовсе – на него по-прежнему ссылается сам пул, поэтому произвольно долго переживает не только удаление записи из словаря приложения, но и принудительную полную сборку мусора. Тот факт, что оба маркера – предназначенный для шифрования и предназначенный для хранения в открытом виде – обнаружены с одинаковой кратностью (по три вхождения), подтверждает эту гипотезу: искать нужно не в структурах данных приложения (там разница есть – она в шифровании), а в разделяемой инфраструктуре, которая одинаково обрабатывает оба вида данных на пути от сети до кода приложения и обратно.

Выводы

В ходе выполнения лабораторной работы существующее приложение **SecureApp** было расширено разделом, работающим исключительно с оперативной памятью процесса, без какого-либо постоянного хранения, и поддерживающим создание, обновление и удаление как конфиденциальных (шифруемых непосредственно при вводе), так и неконфиденциальных данных. Выполнен анализ кода на предмет обработки некорректного ввода (семь проверок, ни одна не привела к падению или зависанию процесса), на предмет потокобезопасности и на предмет обращения с конфиденциальными данными в памяти (добавлена явная, но осознанно ограниченная – в силу неизменяемости строк в .NET – очистка собственных промежуточных буферов). Проведён анализ потребления процессорного времени и памяти под нагрузкой средствами **dotnet-counters**, а также снято и проанализировано три дампа оперативной памяти – после создания, после обновления и после удаления данных, – с прямым поиском в них как конфиденциальных, так и неконфиденциальных контрольных значений.

Семантически работа продемонстрировала принципиальную границу между тем, что может гарантировать код приложения, и тем, что определяется средой выполнения и фреймворком: приложение корректно не хранит конфиденциальные данные в открытом виде ни в одной из своих собственных структур – это подтверждено на уровне кода, – однако открытый текст тем не менее был обнаружен в дампе процесса на всех трёх контрольных точках, включая дампы, снятые после удаления записи и принудительной полной сборки мусора. Это не опровергает корректность реализованного шифрования, а показывает границы его применимости: шифрование защищает данные там, где код приложения ими управляет (в частности – при попадании на диск, что было отдельно проверено в лабораторной работе №2.1), но не может защитить от копий, создаваемых инфраструктурой (буферизация HTTP-запроса и ответа) ещё до и после того, как код приложения получает управление. Разработчик, полагающийся исключительно на факт вызова функции шифрования, рискует сделать ложный вывод о защищённости данных в памяти процесса – вывод, который не подтверждается прямым наблюдением дампа.

С прагматической точки зрения работа показала, что стандартные инструменты диагностики .NET (**dotnet-dump**, **dotnet-gcdump**, **dotnet-counters**) – точные функциональные аналоги перечисленных в задании средств JDK (**jvisualvm**, **jstack**, **jhat**) – достаточны для содержательного криминалистического анализа состояния оперативной памяти реального веб-приложения без установки стороннего специализированного ПО, а методика поиска контрольного маркера в необработанном дампе утилитой **strings** (аналогичная применённой в лабораторной работе №2.1 к

файлу базы данных) одинаково хорошо работает и для файла, и для дампа процесса. Также показано, что для защиты конфиденциальных данных в памяти управляемой среды нельзя полагаться на класс-обёртку вроде **SecureString** – обоснование его неприменимости, а не просто его отсутствие, само по себе является частью анализа безопасности кода.